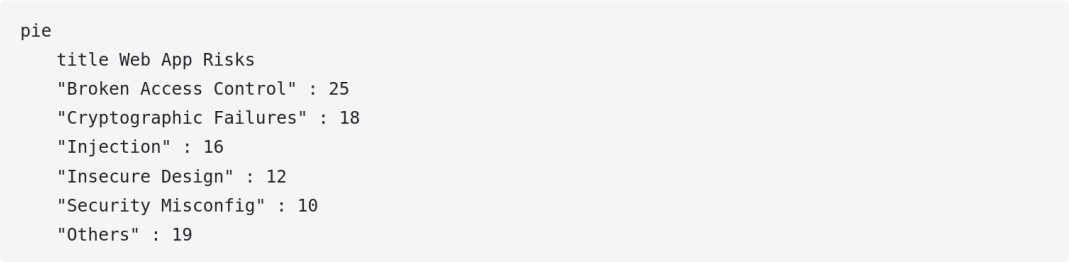


Cybersecurity Training Diary

Day 19: Secure Coding & OWASP Top 10

1. Topics Covered

✓ OWASP Top 10 (2024) Vulnerabilities



✓ Secure Coding Practices

Vulnerability	Bad Practice	Secure Fix
SQL Injection	<code>"SELECT * FROM users WHERE id = " + userInput</code>	<code>db.execute("SELECT * FROM users WHERE id = ?", [userInput])</code>
XSS	<code><div>\${userComment}</div></code>	Sanitize output: <code><div>\${escapeHtml(userComment)}</div></code>
Hardcoded Keys	<code>String apiKey = "12345";</code>	<code>String apiKey = System.getenv("API_KEY");</code>

✓ SAST vs DAST Tools

Type	When Used	Tools
SAST	During development	SonarQube, Checkmarx
DAST	In production	OWASP ZAP, Burp Suite

2. Key Takeaways

- ✓ **Shift Left:** Fix security issues early in SDLC (costs 100x less)
- ✓ **Input Validation ≠ Output Encoding:** Need both for XSS prevention
- ✓ **OWASP Cheat Sheets** are gold for developers

3. Practical Exercises

♦ Exercise 1: Fix Vulnerable Code (JavaScript)

```
// Vulnerable:
app.get('/user', (req, res) => {
  db.query(`SELECT * FROM users WHERE id = ${req.query.id}`);
});

// Fixed:
app.get('/user', (req, res) => {
  db.query("SELECT * FROM users WHERE id = ?", [req.query.id]);
});
```

♦ Exercise 2: Run OWASP ZAP Scan

```
docker run -t owasp/zap2docker zap-baseline.py \
-t https://example.com
```

4. Questions for Reflection

- ? Why do SQL injections still exist in 2024?
- ? How would you convince developers to prioritize security?